Automated Copywriting & Tone Transformer

DecodeLabs GenAI Track — Project 2. Takes a raw product description and generates professional, platform-tailored marketing copy, with full control over the model's creative variance.

Architecture

```
CLI (argparse)  →  Prompt Compiler (hidden master template)  →  Router
                                                                  ├─ Real-time: a
                                                                  └─ Bulk: OpenAI
                                                                              ↓
                                                                 Pydantic valida
```

- **`src/models.py`** — Pydantic schemas (`CopyOutput` , `GenerationRequest`) and per-platform hard limits (char count, hashtag rules).
- **`src/templates.py`** — the hidden master prompt template. Raw variables (`Product_Name` , `Platform` , `Tone`) are injected via f-string compilation; this is the only place allowed to shape the actual model instruction.
- **`src/config.py`** — routes `temperature` / `top_p` and picks the correct token-limit parameter name (`max_tokens` for legacy models like `gpt-4` , `max_completion_tokens` for reasoning models like `o1` / `gpt-4o`).
- **`src/engine.py`** — async real-time pipeline: a semaphore caps concurrent in-flight requests (default 10) to avoid HTTP 429s, and failed calls retry with exponential backoff + jitter.
- **`src/batch.py`** — bulk pipeline for jobs that don't need live UI streaming: builds the JSONL body for the OpenAI Batch API, with an optional `openbatch` fast-path if that package is installed.
- **`src/mock_client.py`** — offline fake client so you can run and demo the whole pipeline without an API key.

Setup

```
pip install -r requirements.txt
```

```
export OPENAI_API_KEY=sk-...        # skip this if you're using --mock
```

Usage — real-time (single product, multiple platforms)

```
python run.py \
  --product "Aurora Desk Lamp" \
  --description "A dimmable LED desk lamp with a wireless charging base and USB-C
  --platform linkedin instagram email twitter \
  --tone witty \
  --temperature 0.8 --top-p 0.9 \
  --out results.json
```

Run it offline with no API key using the mock client:

```
python run.py --product "Aurora Desk Lamp" --description "..." \
  --platform linkedin instagram twitter --tone witty --mock
```

Flags:

Flag	Meaning
`--platform`	one or more of `linkedin instagram email twitter`
`--tone`	`professional witty urgent friendly luxury bold`
`--temperature`	0.0 (consistent/factual) → 2.0 (max variance)
`--top-p`	nucleus sampling cutoff
`--model`	any OpenAI model name; token param is auto-routed
`--concurrency`	max simultaneous in-flight requests

Usage — bulk (CSV of many products/platforms)

```
python -m src.batch examples/products.csv --out batch_input.jsonl --submit
```

Drop `--submit` to just generate the JSONL file for manual review before sending it to OpenAI. CSV columns: `product_name,product_description,platform,tone[,model]` — see `examples/products.csv`.

Why the dual-pipeline design

A synchronous loop hitting the API once per product doesn't scale — each call blocks on network I/O. The async engine overlaps those waits so 10 products finish in roughly the time of the slowest single call, not the sum of all of them. But if you're generating 5,000 product variants and don't need the result back instantly, the Batch API is strictly better: half the token cost, a separate rate-limit pool, at the cost of a 24-hour SLA instead of seconds. This project routes to whichever fits the job rather than forcing everything through one path.

Compliance layer

Every generated `CopyOutput` runs through `compliance_check()` before it's returned — character-count and hashtag-count validation against the target platform's actual limits (e.g., 280 chars for X/Twitter). A generation that violates its own platform's constraints is treated as a failed call and retried, not silently returned.